# Transformers and Self-Attention

*Core concepts behind modern language models*

## Purpose

This short reference note is written for learning, semantic retrieval, and grounded question-answering experiments. It uses original explanatory content and is organized into coherent sections rather than unrelated filler.

## 1. Why Transformers Matter

A language model needs to interpret relationships between words and tokens. In the sentence “The animal did not cross the road because it was tired,” the word “it” refers to the animal. A useful model must combine information from different parts of the sequence. Earlier neural architectures often processed tokens sequentially, while the Transformer architecture uses attention to relate tokens directly to one another. This makes parallel processing during training practical and helps the model represent long-range dependencies.

## 2. Tokens, Embeddings, and Positions

Text is first converted into tokens. A token may be a whole word, part of a word, punctuation, or another text unit. Each token ID is mapped to a vector called an embedding. Since attention alone does not inherently encode token order, positional information is added to the token representations. The resulting vectors carry both lexical information and a signal about where each token appears in the sequence.

## 3. The Self-Attention Operation

Self-attention creates query, key, and value vectors from each token representation. A query represents what a token is looking for; keys represent what other tokens offer for matching; values carry the information that can be combined. Similarity scores between queries and keys are scaled and normalized with softmax. The normalized weights are used to compute a weighted combination of values. In simplified notation, Attention(Q,K,V) = softmax(QK■ / √d■)V. The scaling term helps keep scores in a useful range as the key dimension grows.

## 4. Multi-Head Attention and Layers

Multi-head attention runs several attention operations in parallel. Different heads can learn different relationships, such as local syntax, coreference, or broader semantic associations. Their outputs are combined and projected. Transformer blocks typically also include feed-forward layers, residual connections, and layer normalization. Stacking blocks allows representations to become progressively more contextual. The exact architecture differs between encoder-only, decoder-only, and encoder–decoder models.

## 5. A Small Example

Consider “A student opened the notebook because the lecture had started.” To interpret “the lecture,” the model can attend to “started”; to interpret “the notebook,” it can attend to “opened” and “student.” These are illustrative relationships, not guaranteed interpretations of particular attention heads. Attention weights are useful internal computations, but they should not automatically be treated as complete explanations of a model’s reasoning.

## 6. Practical Strengths and Limits

Transformers support parallel training and flexible context mixing, but standard full self-attention has computational and memory costs that grow approximately quadratically with sequence length. Long documents therefore require careful context management, chunking, or specialized attention methods. Attention also does not guarantee factual correctness: a model may produce a fluent answer that is unsupported by its input.

## Key Takeaways

Transformers build contextual token representations using attention and stacked neural-network blocks. Tokenization, embeddings, positional information, and attention work together. Understanding these components helps explain why context length, chunk boundaries, and input quality matter in retrieval-augmented generation systems.

End of document • Use source passages as evidence and verify claims during evaluation.